

Definição e Justificativa da Stack de Interface

1. Perfil de Dispositivo e Público-Alvo

A escolha das tecnologias de interface foi diretamente guiada pelo perfil de dispositivo dos usuários que acessarão o sistema. De acordo com o documento de visão e escopo técnico:

- **Dispositivo-alvo:** O foco principal de acesso é via **PC** (desktop e notebook).
- **Justificativa de Uso:** Os perfis de **Administrador**, **Engenharia** e **Financeiro** trabalham majoritariamente no escritório ou em campo utilizando notebooks. O uso dessas ferramentas exige telas maiores, pois a operação diária envolve o preenchimento de formulários e a manipulação intensiva de tabelas e cronogramas detalhados.
- **Responsividade (Acesso Mobile - RNF-01):** O sistema deve possuir layout responsivo para acessos ocasionais via celular. O sistema garantirá usabilidade em resoluções a partir de **360px** (smartphones) até **1920px** (monitores Full HD).
Atenção à restrição de scroll: A página principal não apresentará barras de rolagem horizontais globais ("indesejadas"). Para componentes de alta densidade de dados, como o Gráfico de Gantt e o Kanban, será adotado o padrão de mercado de *scroll horizontal interno* restrito ao contêiner do componente, garantindo a legibilidade em telas menores sem quebrar o layout da página.
- **Exceção de Campo:** As equipes de instalação (trabalhadores em campo) não terão acesso ao sistema completo. Para estes, será gerado apenas um **link mobile simplificado** com a lista de materiais e botão de rota.

2. Tecnologias Escolhidas e Justificativas

A base da stack já foi padronizada na etapa de prospecção, definindo o uso de **React** para o frontend em conjunto com **C#** no backend. Para atender às demandas específicas (Kanban e Gantt), definimos o seguinte ecossistema ao redor do React:

2.1. Core da Aplicação

- **React (via Vite):** Framework principal definido para o projeto. O uso do Vite garante um ambiente de desenvolvimento extremamente rápido e *builds* otimizados, ideal para aplicações web (*Single Page Applications*) que não dependem de servidor local.
- **React Router Dom:** Essencial para o gerenciamento de rotas e proteção de telas de acordo com o perfil de acesso do usuário, atendendo ao requisito de **Controle de Permissões (RBAC)**.

2.2. Estilização e Acessibilidade (Atendimento ao RNF-01 e RNF-05)

- **Tailwind CSS:** Framework de CSS utilitário que permite construir interfaces altamente responsivas com rapidez. Ele será a peça-chave para garantir que a interface funcione nas resoluções exigidas (**360px a 1920px**).
- **shadcn/ui (baseado em Base UI, com suporte alternativo a Radix UI):** Biblioteca de componentes acessíveis e sem estilo predefinido, que se integra nativamente ao Tailwind. Atende à necessidade de construir uma interface utilizável por perfis não técnicos (**Engenharia e Financeiro**) de forma rápida e padronizada.

2.3. Bibliotecas Específicas de Negócio

- **DHTMLX Gantt:** Para a construção do **Gráfico de Gantt**, ferramenta sugerida para permitir a alocação de equipes por data, reordenação manual de prazos via drag-and-drop.
 - **Decisão Arquitetural sobre Propagação de Atrasos:** Como o recurso de "auto-scheduling" (propagação automática de datas) é um recurso pago em bibliotecas comerciais, a lógica de "arrastar uma tarefa e empurrar as sucessoras ignorando finais de semana" será **desenvolvida customizada pela equipe** (no frontend ou via algoritmo no backend), limitando-se à complexidade de dependências simples do tipo "fim-início".
 - A cascata desloca as tarefas sucessoras em unidades de dia inteiro, pulando finais de semana e feriados; o horário (hora:minuto) de cada tarefa é armazenado e exibido, mas preservado, não é recalculado automaticamente pela propagação, só uma edição manual do usuário o altera.

- **dnd-kit:** Ferramentas modernas de *Drag and Drop* para criar o componente do **Funil de Obras (Kanban)**. Será utilizado para permitir a movimentação fluida dos cards entre as colunas "*Material Comprado*", "*No Depósito*", "*Separado*", "*Em Andamento*" e "*Concluído*".
- **Gerenciamento de Estado Local (Cliente):** Zustand será usado exclusivamente para estado de interface sem origem no servidor. Filtro selecionado, aba ativa, modal aberto, item sendo arrastado no momento, etc. Dados de negócio (tarefas do Gantt, cards do Kanban) não residem no Zustand; sua origem, cache e sincronização em tempo real são responsabilidade do React Query.

2.4. Formulários e Validações

- **React Hook Form + Zod:** Utilizados para construir os formulários de cadastro de **Obras** e **Clientes**. Garantem que as validações de campos obrigatórios funcionem com alta performance e sem *re-renderizações* desnecessárias na tela.

2.5. Comunicação, Cache e Autenticação

Para garantir uma integração de alta performance entre a interface React e a API em C#, a stack utilizará os seguintes padrões:

- **Axios:** Ferramenta para realizar as requisições HTTP (REST API), configurada com interceptadores para injetar automaticamente o token JWT nas chamadas, garantindo a comunicação segura.
- **React Query (TanStack Query):** Fonte única da verdade para todo dado de origem no servidor (tarefas do Gantt, cards do Kanban e demais entidades de negócio), incluindo cache e estados de carregamento (isLoading), evitando a criação manual dessa lógica em cada tela.
 - **Sincronização em Tempo Real (RF-10):** Quando o backend disparar um evento via SignalR (ex.: início de uma atividade no Gantt refletindo no status do Kanban), um handler dedicado no frontend traduz esse evento em uma chamada a *invalidateQueries* (ou *setQueryData*, quando a atualização direta for mais eficiente) do React Query, que então atualiza silenciosamente os dados na interface. Essa integração não é automática — cabe à equipe implementar o handler que liga o evento do SignalR à invalidação do cache.
- **Gestão de Sessão com JWT:** Atendendo ao requisito de Autenticação (RF-01), o frontend armazenará o token JWT gerado pelo backend, que servirá de base para a proteção de rotas no React Router Dom e restrição visual de botões conforme o perfil de acesso (RBAC - RF-02).